Exploring Neural Networks for Predicting Next-Pitch Statistics in the MLB

Lindsay White
Batten College of Engineering and Technology
Old Dominion University
December 13, 2022

***Abstract*—With the complexity of AI and legal sports betting on the rise, the application of machine learning models to sports statistics and predictions has become increasingly relevant. This paper looks at how both shallow neural networks and deep neural networks can be used to predict the next pitch in an MLB game, given the game situation. Using pitch data from the complete 2015 season, the model takes into account the count, outs, runners, and inning to make a prediction. This paper will explore the modifications made to the raw data, the comparison of shallow and deep learning models, and the process that led to each model for accurate prediction using Microsoft Excel and MATLAB. This model was able to predict around 41% of pitches of the testing dataset, which is less than researched models for pitch prediction but greater than the baseline accuracy. Further sensitivity studies are needed to determine which game situations are most significant in affecting the overall accuracy of the model.**

***Index Terms*—baseball, deep learning, machine learning, MATLAB, neural networks, shallow learning**

I. Introduction

On November 1st, 2017, the city of Houston celebrated as their baseball team, the Astros, won their first World Series. However, the celebration turned to disappointment as it later came out that the Astros were using cameras to steal signs of the next pitch for the batters [12]. For major league batters, knowing the pitch makes a huge difference. Not only can it help them determine where the pitch might be, but it can help to determine how fast it will be coming and how early they will need to swing. Since most pitches are coming at them around 90mph, being prepared beforehand gives them a big advantage.

In an attempt to formulate a *legal* method for next-pitch prediction, the paper explores multiple classification models. The first, a Naive Bayesian classifier. The second model is a shallow learning model - Support Vector Machines (SVMs). The last two are deep learning models - Convolutional Neural Network (CNN) and a Feed Forward Network (FFN). With each modification made to the neural networks, the accuracy remains the same, landing on a maximum accuracy of around 41%.

This is not the first attempt made at pitch prediction. Other studies have used stacked long term and short-term memory models (LSTM), logistic regression, and multi-class linear discriminant analysis for pitch prediction. The paper will conclude with a comparison of these models to the ones outlined in the following sections.

II. Model

A. Variables

This model takes into account 9 game situation variables: score difference for the pitcher, ball count, strike count, outs, runners on first, second, or third, pitch number per batter, and half inning. Half innings are used as opposed to innings to indicate the top or bottom of the inning.

The resulting classes are 18 different pitches: Four-Seam Fastball (FF), Curveball (CU), Cutter (FC), Sinker (SI), Changeup (CH), Two-Seam Fastball (FT), Intentional Ball (IN), Slider (SL), Knuckle Curve (KC), Eephus (EP), Splitter (FS), Pitchout (PO), Knuckleball (KN), Screwball (SC), and unknown (UN, AB, FA). However, given that a handful of these pitches are rare and/or similar to another, they were condensed into the classes outlined in Table 1 with the help of baseball experts [11].

TABLE I
Breakdown of Condensed Pitch Classes

Pitch Classes	
Four-Seam Fastball	FF, FC
Two-Seam Fastball	SI,FT
Splitter	FS,FA
Slider	SL
Changeup	CH
Curveball	CU, KC
Intentional Ball	IN, PO, FO
Unknown and Uncommon	EP, AB, KN, SC, UN

B. Data

The data used in this model was obtained from a data set on Kaggle. However, the raw data required many modifications before being able to run through a model. The first issue was that the data file was much larger than Excel can fit. The data is supposed to have around 2.7 million lines of data from 2015 through 2018. However, Excel's limit is $1,048,576$, so this model uses only 2015 data.

Once broken apart, the data came along with a lot of extraneous data that needed to be filtered out. Pitch data included location, speed, and spin of the ball, which can be used in a more complicated model. Next, the data required columns for inning and score differential. To get the half innings, outs and event numbers could be used to create an algorithm for

Excel to understand, resulting in 17 or 18 half innings rather than the standard 9. For the score, we needed to know the difference rather than just the score of the batting team which was provided. To do this, IF statements are nested in Excel to build a column for the pitching team's score, and ultimately the score differential. Any lines without a class were deleted. For the naive Bayesian classifier, shallow learning model, and feed forword network, each pitch code was given a numeric value (e.g. 1 for four-seam fastball, 2 for two-seam fastball, etc.). For the CNN model, the pitch codes were converted to categorical data. Finally, the files were converted to .csv files for ease of uploading into MATLAB.
Some models don't take well to zeros as an input and may delete an entire row or column if a zero is detected. To reduce this risk, all the input values are normalized between 1 and 2.

III. Methods and Experiments

Before exploring how shallow and deep learning can be used to predict pitches, we first look at a probabilistic classifier to show the need for a neural network. The baseline accuracy we are looking to improve is 12.50%, which is the accuracy if a random pitch it guessed - one in eight.

A. Naive Bayesian Classifier

As a starting point, we begin with the Naive Bayesian Classifier. This model assumes conditional independence. In other words, conditional probabilities are simply multiplied to find the posterior probability without accounting for dependence of input variables. The 2015 data was split by 70% for training and 30% for testing data. Equation 1 is used to find the posterior probability.

$$P(C_k|x) = P(x_1, x_2, ..., x_D|C_k) \cdot P(C_k) \tag{1}$$

where D is the dimensions of the data and k is the number of classes. Each individual likelihood can be calculated assuming a Gaussian distribution. The Gaussian distribution is computed using the following equation:

$$Norm(x|\mu, \sigma^2) = \frac{1}{(2\pi\sigma^2)^{(0.5)}} exp\Big\{\frac{-1}{2\sigma^2}(x-\mu)^2\Big\} \tag{2}$$

where μ represents the mean of each class and σ^2 represents the variance for each class.
We can predict what the ID is by choosing the column with the greatest probability for each vector. However, this model has a few flaws. First, assuming conditional independence is not realistic in this model, as there are inputs that affect each other. For example, the score may affect a batter to swing more or less, changing the pitcher's tactic. Another flaw is that it does not contain layers as in neural networking for training to classify unfamiliar data. Ultimately, Naive Bayesian classifier outputs an accuracy of 24.25% even after complete data modification. Thus, predicting a pitch requires a more complex model.

B. Support Vector Machines

First, we take shallow neural networks and look at Support Vector Machines (SVMs). SVMs are models that aim to maximize the smallest distance between the decision boundary and any datapoint [1]. To carry this out in MATLAB, one-versus-one coding must be used to create a binary column for each class, where a 1 is only used when a line is classified as that class. In this method, the classifier uses $\frac{m(m-1)}{2}$ SVMs where m is the number of classes.
MATLAB has a built in function *fitcauto* that will automatically find the type of classification model and optimal hyperparameters that best fits each iteration. For this model, it may choose ensemble, net, nb, or tree. 70% of the data was used for training and 30% used for testing, randomly selected rows of data. This model must also be partitioned to avoid overfitting.
As a result, the SVMs gives an accuracy of 40.92%, which is greater than the probabilistic model and baseline. However, with 9 input features, a deep learning network may be preferable.

C. Convolutional Neural Network

The first deep learning model used for pitch classification is CNN. Training and testing data were split up randomly, setting aside 30% for testing. The architecture must be constructed by hand since pre-trained models for numerical data classification are scarce. The following layers were used:

featureInputLayer(numFeatures,"Name","input")
convolution1dLayer(10,12,"Name","conv1")
fullyConnectedLayer(100,"Name","fc1")
batchNormalizationLayer("Name","bn1")
leakyReluLayer("Name","relu")
fullyConnectedLayer(50,"Name","fc2")
batchNormalizationLayer("Name","bn2")
leakyReluLayer("Name","relu2")
fullyConnectedLayer(numClasses,"Name","fc3")
softmaxLayer("Name","sm")
classificationLayer

This architecture begins with an input layer for features as opposed to images, moves to the convolutional layer, then goes through a few layers of fully connected layers, being sure to normalize the batch. The convolutional layer can have the feature numbers adjusted, and was found to give optimal results with the given values. A leakyReLu layer is also included to address the vanishing gradient problem as well as the dying ReLU problem, discussed in the "Results" section. In the end, we convert the numbers to probabilities using the softmax layer. The layers were kept simple in order to avoid overfitting as best as possible.

The model stabilizes after about only 5% of the total iterations, but unfortunately, it stabilizes around 40%. Figure 1 displays the training graph for the model. On average, this deep learning model yields a training accuracy of 41.05% and a testing accuracy of 40.52%.

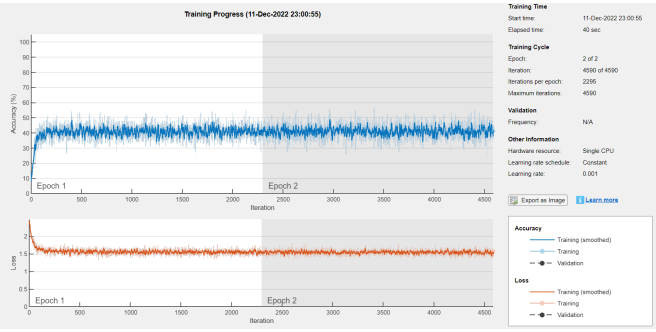


Fig. 1. CNN Training Graph

However, this model also contains flaws, the main one being that that CNN is designed to reduce the number of inputs, and with a model that already only has 9 one-dimensional inputs, running through a CNN affects a large portion of the data. CNN would be a better model for image classification as opposed to feature inputs.

D. Feed Forward Network

Finally, a second deep network is explored - feed forward network (FFN). This is a multilayer neural network that contains only a specified number of hidden layers between the input and output, seen in Figure 2 [7]. Thus, it is more complex than a shallow learning model, but does not reduce inputs like the CNN.

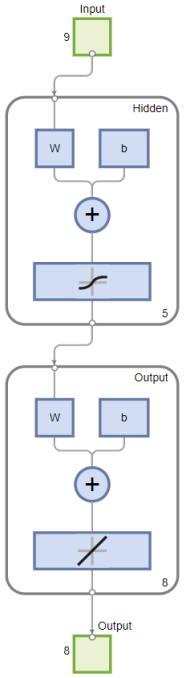


Fig. 2. Feed Forward Network Diagram

This network uses a sigmoid activation function, $sigmoid[0, 1]$. The data has been normalized and the goal is to output a probability of which class the data belongs to.

FFN is unable to compute categorical data, so all classes must be converted to a numerical value. However, because of this, the trained network uses a continuous optimization function (outputs are decimals such as $2.304, 3.938$, etc.). Because we are only looking for discrete classes, we must utilize one-versus-one coding.

Multiple trials were done, and it was found that 5 was the optimal number of hidden layers for this model. FFN in MATLAB automatically breaks the data into training, validation, and testing. These errors are laid out in Figure 3. The resulting mean square error is 9.17%.

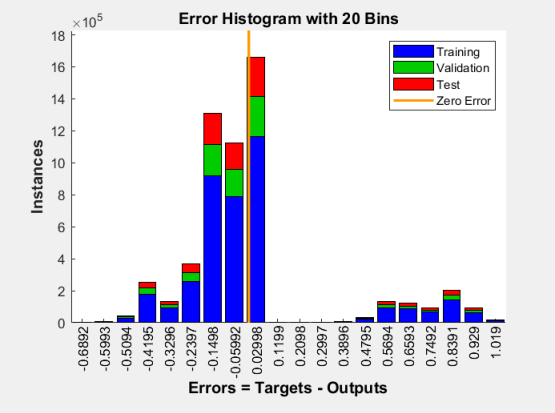


Fig. 3. Feed Forward Network Error Histogram

FFN amounts to an average accuracy of 41.49%.

E. Results

This study looked at a probabilistic classifier, a shallow learning model, and two deep learning models. The accuracy results are summarized in Table 2.

TABLE II
SUMMARY OF MODEL RESULTS

Accuracy Results	
Baseline	12.50%
Naive Bayesian Classifier	24.25%
Support Vector Machine	40.92%
Convolutional Neural Network	40.52%
Feed Forward Network	41.49%

For the SVM and Feed Forward Network, it was beneficial to include a lower training rate with more epochs. Training rate determines how quickly the model will adjust to fit the data. However, for the CNN, because the model included so many layers, it was more beneficial to run the model with a higher learning rate so it could take more time to run through each layer.

SVM and FFN models also outputted a training accuracy of 100% despite the testing accuracies around 40%. That means this model is most likely overfitting to classify each piece of data.

One challenge with this model is the optimization algorithm. The default setting for the numerical models, SVM and FFN, is a continuous function optimization. That is why the FFN originally returned optimal pitches of things like 2.312 and the like. Part of the challenge of this model is implementing combinatorial optimization, which classifies to discrete variables.

There are a few factors that contribute to a consistently poor performance from the models. The biggest variable is that a large portion of the data belongs to one specific class. The confusion matrix in Table 3 depicts how each piece of data was classified in the most accurate model, the FFN.

TABLE III
FFN CONFUSION MATRIX

	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**
1	288063	0	0	0	0	0	471	0
2	148854	0	0	0	0	0	256	0
3	11214	0	0	0	0	0	13	0
4	101814	0	0	0	0	0	100	0
5	73916	0	0	0	0	0	103	0
6	68797	0	0	0	0	0	41	0
7	2822	0	0	0	0	0	775	0
8	4036	0	0	0	0	0	6	0

It is clear that in this situation, Four-Seam Fastball (Class 1) and Intentional Ball (Class 7) are the most dominant classes. As a result, all misclassifications belong to the Four-Seam Fastball or Intentional Ball. Models with one or two dominant classes have a hard time classifying the less common classes and result in decreased accuracy. Possible solutions for this are discussed in the "Future Work" section.

Another explanation for low accuracy is the use of one-versus-one coding. Although in some models it was necessary, one-versus-one coding for multiclass problems, especially in SVMs, holds no guarantee for correct scales, thus introducing error.

Additionally, the CNN model runs the risk of the dying ReLu problem [3]. This is when a large gradient flowing through a neuron becomes zero and causes the neuron to permanently "die", or become deactivated. This can be combatted with a lower learning rate or a leakyReLu which adds a small gradient to negative inputs. However, it may be impossible to completely solve this issue as there is still the risk of dying neurons through the ReLu layer.

F. Discussion

Machine learning in the sports realm is a hot topic today. Thus, it should not come as a surprise that several other studies have been performed on this dataset and similar ones.

The first study by Glenn Sidle and Hien Tran uses multiclass classification methods to predict the next pitch [8]. This study broke up each model by individual pitcher, including only those who threw at least 500 pitches in two consecutive seasons. This is a key element as specific pitchers have tendencies toward certain pitches. One pitcher might only throw curveballs to lefties while another loves his fastball. Additionally, this model considered 29 features where our model considers 9. Things like time of day and previous pitch were taken into account. Increasing the number of features can help to increase the accuracy, so long as there is a correlation. Overall, this model yielded an average accuracy of 65.40% between three models: SVMs, classification trees, and linear discriminant analysis. This accuracy is larger than our model given the increased complexity.

Another previous study done by Christopher Watkins looks at multinomial logistic regression and SMVs for predicting the next pitch [11]. This model classifies only four different types of pitches compared to the 8 in our model. This model also only looked at pitchers that threw over 1000 pitches in each of the 2011-2013 seasons. This is a great way to ensure the pitchers being used are reliable and consistent. Overall, the Watkins model produced 74.5% accuracy for all pitches. This is due to the decreased number of classes, making it easier to classify, and the selection of pitchers, making the model more reliable.

The third researched model from Chih-Chang Yu, Chih-Ching Chang, and Hsu-Yung Cheng shows how stacked long term and short-term memory model (LSTM) can be used for pitch prediction [2]. LSTM is used after the robust neural network to help solve the vanishing gradient problem. Yu, Chang, and Cheng use sequential pitches for prediction, stating that a pitcher facing a batter looks more like a sequence than individual pitches. In addition, they classify 13 pitches as opposed to 8.

The most significant factor to the success of this model is the attention mechanism. In other words, if a feature is more important, it will have more attention in the prediction model. This is how the human brain works, so it is a way to improve the model. Ultimately, this model yielded 76.00% pitch prediction accuracy.

G. Future Work

Looking into related studies as well as evaluating our model, there are a few takeaways for improvement of this model in the future:

- Tackle the issue of the dominating pitches by breaking up the model by individual pitchers and/or only looking at consistent pitchers (over a certain number of pitches per season). This will help with a more accurate spread of pitches.
- Another way to handle the imbalanced data is to adjust weights so smaller weights are given to dominating classes, and vice versa. Downsampling can also be implemented to large datasets to manually adjust the number of classes or noise can be added. Another option is to generate artificial data using SMOTE to balance the classes.
- Explore options for weighing the outcomes in reference to an attention mechanism. This will help the model to pay attention to the features that matter most.
- Consider an increase in features and/or a decrease in pitch type. Features with the biggest impact include those

involving pitcher-batter match-up such as handedness and previous at-bats.
- Add a mechanism for looking at sequential pitches and how that affects the following pitch.

H. Conclusion

The model discussed in this paper provides a solid foundation for further improvement. We can rule out a probabilistic model for pitch classification based on the results. We can also conclude that having a few dominant pitches has a big impact on the results of the prediction. Although, consistency in a model is important, and yielding around 40% in every neural network attempt gives stable results.

The future of this model will include a reevaluation of both the features and classes. The data itself will also be improved, as other works with better results trained their models using data from multiple MLB seasons.

Overall, a yield of 41% accuracy is not a bad model, but there is much room for improvement in the future.

REFERENCES

[1] Aggarwal, Charu C. Neural Networks and Deep Learning. Springer, 2018.

[2] C. -C. Yu, C. -C. Chang and H. -Y. Cheng, "Decide the Next Pitch: A Pitch Prediction Model Using Attention-Based LSTM," 2022 IEEE International Conference on Multimedia and Expo Workshops (ICMEW), 2022, pp. 1-4, doi: 10.1109/ICMEW56448.2022.9859411.

[3] "Convolutional Neural Networks for Visual Recognition." Stanford, cs231n.github.io/neural-networks-1/ actfun. Accessed 11 Dec. 2022.

[4] "Feedforwardnet." MathWorks, www.mathworks.com/help/deeplearning/ref/feedforwardnet.html. Accessed 11 Dec. 2022.

[5] "List of Deep Learning Layers." MathWorks, www.mathworks.com/help/deeplearning/ug/ list-of-deep-learning-layers.html. Accessed 11 Dec. 2022.

[6] "Predict." MathWorks, www.mathworks.com/help/stats/ classreg.learning.classif.compactclassificationsvm.predict.html. Accessed 11 Dec. 2022.

[7] Schale, Paul. "MLB Pitch Data 2015-2018."

[8] Sidle, Glenn and Tran, Hien. 'Using Multi-class Classification Methods to Predict Baseball Pitch Types'. 1 Jan. 2018 : 85 – 93.

[9] "Specify Layers of Convolutional Neural Network." MathWorks, www.mathworks.com/help/deeplearning/ug/ layers-of-a-convolutional-neural-network.html. Accessed 11 Dec. 2022.

[10] "TrainNetwork." MathWorks, www.mathworks.com/help/deeplearning/ref/trainnetwork.html. Accessed 11 Dec. 2022.

[11] Watkins, C. (2020). Novel Statistical and Machine Learning Methods for the Forecasting and Analysis of Major League Baseball Player Performance (Order No. 27964705). Available from ProQuest Dissertations Theses Global. (2406520816).

[12] Weese, Lukas. "Astros cheating scandal timeline: A look at Houston's sign-stealing controversy." The Athletic.

[13] "What Is Deep Learning?" MathWorks, www.mathworks.com/discovery/deep-learning.html. Accessed 11 Dec. 2022.